

Ladegerät-Zustandsmodell in Java

State-Pattern, Polymorphie, abstrakte Klassen und Singleton/Factory

Benjamin Malicke ©

2. März 2026

Inhaltsverzeichnis

1	Einführung	2
2	Warum State-Pattern? (Vorher/Nachher)	2
3	Abstrakte Klassen und Methoden (Wiederholung)	2
3.1	Abstrakte Basisklasse Zustand	3
4	Die Klasse Ladegeraet und Polymorphie	3
4.1	Code der Klasse Ladegeraet	3
5	Singleton-Pattern und einfache Factory-Idee	4
6	Konkrete Zustandsklassen (mit didaktischem Logging)	4
6.1	NichtLadend	4
6.2	NormalLadend	5
6.3	SchnellLadend (optional in der Demo)	5
7	Beispielhafter Ablauf (Main) und Sichtbarkeit der Polymorphie	6
7.1	Main-Klasse Ladestation	6
8	Mini-Übungen (Wiederholung & Festigung)	7
9	Zusammenfassung (Takeaways)	7
10	Lösungen zu den Mini-Übungen	8

1 Einführung

In diesem Dokument wird ein kleines Java-Projekt beschrieben, das das Verhalten eines *Ladegeräts* mit dem **State-Pattern** modelliert. Dabei wiederholen und vertiefen wir zentrale objektorientierte Konzepte:

- **Abstrakte Klassen** und **abstrakte Methoden**
- **Polymorphie** (dynamisches/spätes Binden)
- **Singleton-Pattern** und die einfache **Factory-Idee** über statische Getter-Methoden

Lernziele (Checkliste)

Nach dem Durcharbeiten kannst du:

- erklären, warum das **State-Pattern** lange `if/switch`-Ketten ersetzt,
- **abstrakte Klassen** und **abstrakte Methoden** definieren und begründen, warum sie im State-Pattern sinnvoll sind,
- **Polymorphie** am Aufruf `zustand.bearbeiten(ladegeraet)` beschreiben (Compile-Zeit-Typ vs. Laufzeit-Typ),
- erläutern, warum bei **Singletons** der Konstruktor `private` ist und wie eine **Factory-Idee** (statische Getter) Instanzen liefert.

2 Warum State-Pattern? (Vorher/Nachher)

```
1 if (ladestand < 20) {
2     // schnell laden
3 } else if (ladestand < 100) {
4     // normal laden
5 } else {
6     // nicht laden
7 }
```

```
1 // ladegeraet.hat einen Zustand (polymorph)
2 ladegeraet.ausloesen(); // delegiert an zustand.bearbeiten(ladegeraet)
```

Vorteile: zustandsspezifisches Verhalten ist gekapselt, neue Zustände *hinzufügen* statt `if`-Ketten anfassen (Open-Closed-Prinzip). Polymorphie wählt zur *Laufzeit* die passende `bearbeiten(...)`-Implementierung.

3 Abstrakte Klassen und Methoden (Wiederholung)

Merksatz: Abstrakte Klassen können *gemeinsame Implementierung* enthalten und *abstrakte Methoden* vorgeben. Von `abstract` Klassen kann man keine Objekte direkt erzeugen (`new` nicht erlaubt).

3.1 Abstrakte Basisklasse Zustand

Listing 1: Abstrakte Basisklasse Zustand

```
1 package _02_03_2026;
2
3 public abstract class Zustand {
4     public abstract void bearbeiten(Ladegeraet ladegeraet);
5 }
```

Jede konkrete Zustandsklasse (z. B. NichtLadend, NormalLadend, SchnellLadend) *muss* bearbeiten(...) implementieren.

4 Die Klasse Ladegeraet und Polymorphie

4.1 Code der Klasse Ladegeraet

Listing 2: Klasse Ladegeraet

```
1 package _02_03_2026;
2
3 public class Ladegeraet {
4
5     private Zustand zustand;
6     private int ladestand;
7
8     public Ladegeraet(int ladestand) {
9         // Ladestand validieren/klammern (0..100)
10        this.setLadestand(ladestand);
11
12        // Startzustand: NichtLadend (Singleton-Instanz)
13        // (Didaktisch: Initialisierung sichtbar ueber Getter)
14        this.zustand = NichtLadend.getNichtLadend();
15        System.out.println("Init-Zustand: NichtLadend (Singleton-Getter)");
16    }
17
18    public void setLadestand(int ladestand) {
19        // Clamping auf 0..100 zur Robustheit
20        this.ladestand = Math.max(0, Math.min(100, ladestand));
21    }
22
23    public int getLadestand() {
24        return ladestand;
25    }
26
27    public void setZustand(Zustand zustand) {
28        this.zustand = zustand;
29    }
30
31    public void ausloesen() {
32        // Polymorpher Aufruf:
33        // zur Laufzeit entscheidet die konkrete Klasse von 'zustand'
34        zustand.bearbeiten(this);
35    }
36 }
```

Polymorphie sichtbar machen

Wir fügen in den Zuständen `println`-Ausgaben ein, damit beim Aufruf von `ausloesen()` klar wird, *welche* `bearbeiten(...)`-Methode tatsächlich läuft.

5 Singleton-Pattern und einfache Factory-Idee

Kernidee: Bei **Singletons** ist der Konstruktor `private`, damit niemand von außen `new` aufrufen kann. Eine **statische Getter-Methode** (*Factory-Idee*) liefert die *eine* Instanz.

- **Privater Konstruktor:** verhindert unkontrollierte Objekterzeugung.
- **Statischer Getter** (z.B. `getNormalLadend()`): gibt beim Aufruf die Singleton-Instanz zurück.
- **Lazy** (erst bei Bedarf erzeugen) oder **Eager** (sofort beim Laden der Klasse) möglich.

6 Konkrete Zustandsklassen (mit didaktischem Logging)

6.1 NichtLadend

Listing 3: Konkreter Zustand NichtLadend

```
1 package _02_03_2026;
2
3 public class NichtLadend extends Zustand {
4
5     private static NichtLadend nichtLadend;
6
7     private NichtLadend() {
8         super();
9     }
10
11     @Override
12     public void bearbeiten(Ladegeraet ladegeraet) {
13         System.out.println("[NichtLadend] pruefen ... Ladestand=" +
14             ladegeraet.getLadestand() + "%");
15         // Didaktische Uebergangslogik:
16         // Bei 20..99% in NormalLadend wechseln (Laden erlaubt)
17         if (ladegeraet.getLadestand() >= 20 && ladegeraet.getLadestand()
18             < 100) {
19             ladegeraet.setZustand(NormalLadend.getNormalLadend());
20             System.out.println(" -> Wechsel zu NormalLadend");
21         } else {
22             System.out.println(" -> Bleibe in NichtLadend");
23         }
24     }
25
26     public static NichtLadend getNichtLadend() {
27         if (nichtLadend == null) {
28             nichtLadend = new NichtLadend();
29         }
30         return nichtLadend;
31     }
32 }
```

6.2 Normalladend

Listing 4: Konkreter Zustand Normalladend

```
1 package _02_03_2026;
2
3 public class Normalladend extends Zustand {
4
5     private static final Normalladend INSTANCE = new Normalladend(); //
        Eager Singleton (einfach + threadsicher genug fuer Demo)
6
7     private Normalladend() {
8         super();
9     }
10
11     @Override
12     public void bearbeiten(Ladegeraet ladegeraet) {
13         int alt = ladegeraet.getLadestand();
14         ladegeraet.setLadestand(alt + 10);
15         int neu = ladegeraet.getLadestand();
16         System.out.printf("[Normalladend] %d%% -> %d%%\n", alt, neu);
17
18         // Didaktik: bei >=100% zurueck in NichtLadend
19         if (neu >= 100) {
20             ladegeraet.setZustand(NichtLadend.getNichtLadend());
21             System.out.println(" -> Akku voll, Wechsel zu NichtLadend");
22         }
23         // Optional: bei sehr niedrigem Stand in Schnellladend drosseln/
            boosten (hier nicht noetig)
24     }
25
26     public static Normalladend getNormalladend() {
27         return INSTANCE;
28     }
29 }
```

6.3 Schnellladend (optional in der Demo)

Listing 5: Konkreter Zustand Schnellladend

```
1 package _02_03_2026;
2
3 public class Schnellladend extends Zustand {
4
5     private static Schnellladend schnellladend;
6
7     private Schnellladend() {
8         super();
9     }
10
11     public static Schnellladend getSchnellladend() {
12         if (schnellladend == null) {
13             schnellladend = new Schnellladend();
14         }
15         return schnellladend;
16     }
17
18     @Override
```

```

19     public void bearbeiten(Ladegeraet ladegeraet) {
20         int alt = ladegeraet.getLadestand();
21         ladegeraet.setLadestand(alt + 20);
22         int neu = ladegeraet.getLadestand();
23         System.out.printf("[SchnellLadend] %d%% -> %d%%\n", alt, neu);
24
25         if (neu >= 100) {
26             ladegeraet.setZustand(NichtLadend.getNichtLadend());
27             System.out.println(" -> Akku voll, Wechsel zu NichtLadend");
28         } else if (neu >= 80) {
29             ladegeraet.setZustand(NormallLadend.getNormallLadend());
30             System.out.println(" -> Ab 80% drosseln: Wechsel zu
31                 NormallLadend");
32         }
33     }

```

7 Beispielhafter Ablauf (Main) und Sichtbarkeit der Polymorphie

7.1 Main-Klasse Ladestation

Listing 6: Beispielhafte Main-Klasse Ladestation

```

1 package _02_03_2026;
2
3 public class Ladestation {
4
5     public static void main(String[] args) {
6         // Ladegeraet mit Start-Ladestand 10%
7         Ladegeraet ladegeraet = new Ladegeraet(10);
8         System.out.println("Start-Ladestand: " + ladegeraet.getLadestand
9             () + "%\n");
10
11         // 1) ausloesen() im Zustand NichtLadend
12         System.out.println("--- Zyklus 1: ausloesen() ---");
13         ladegeraet.ausloesen();
14
15         // Manuell Ladestand aendern (z.B. "von extern" oder zur Demo)
16         ladegeraet.setLadestand(30);
17         System.out.println("\nLadestand manuell gesetzt auf: " +
18             ladegeraet.getLadestand() + "%");
19
20         // 2) ausloesen(): dank vorherigem Wechsel sollte NormallLadend
21         //    laden
22         System.out.println("--- Zyklus 2: ausloesen() ---");
23         ladegeraet.ausloesen();
24
25         // Noch ein paar Zyklen, bis "voll"
26         System.out.println("--- Zyklus 3: ausloesen() ---");
27         ladegeraet.ausloesen();
28         System.out.println("--- Zyklus 4: ausloesen() ---");
29         ladegeraet.ausloesen();
30         System.out.println("--- Zyklus 5: ausloesen() ---");
31         ladegeraet.ausloesen();
32         // usw.
33     }

```

Die `println`-Ausgaben zeigen bei jedem `ausloesen()`-Aufruf, *welcher* konkrete Zustand die `bearbeiten(...)`-Methode ausführt (Polymorphie sichtbar) und wann Zustandswechsel stattfinden.

8 Mini-Übungen (Wiederholung & Festigung)

1. **Neuen Zustand ergänzen:** Implementiere `Erhaltungsladend`, der den Ladestand um 2% erhöht und automatisch aktiv wird, wenn der Akkustand zwischen 95% und 99% liegt. Baue geeignete Übergänge zu `NichtLadend`/`NormalLadend` ein.
2. **Polymorphie beschreiben:** Erkläre in 3–4 Sätzen, was beim Aufruf `ladegeraet.ausloesen()` polymorph passiert. Gehe auf `Compile-Zeit-Typ` vs. `Laufzeit-Typ` ein.
3. **Privater Konstruktor & Factory:** Warum ist bei Singletons der Konstruktor `private`? Wie liefert die statische Getter-Methode dennoch eine Instanz? Nenne Vor- und (mögliche) Nachteile.

9 Zusammenfassung (Takeaways)

- **State-Pattern:** Zustandsabhängiges Verhalten ist in Klassen gekapselt; der Code bleibt erweiterbar (neue Zustände hinzufügen, statt `if`-Ketten zu ändern).
- **Abstrakte Klassen:** Zustand gibt die gemeinsame Signatur (`bearbeiten`) vor, konkrete Zustände implementieren das Verhalten.
- **Polymorphie:** `zustand.bearbeiten(...)` wird zur **Laufzeit** dynamisch gebunden; welches Verhalten ausgeführt wird, hängt vom *tatsächlichen* Objekt ab.
- **Singleton & Factory-Idee:** `private` Konstruktoren verhindern unkontrollierte Instanziierung; statische Getter liefern die Instanzen (eine pro Klasse).

10 Lösungen zu den Mini-Übungen

Aufgabe 1: Neuer Zustand Erhaltungsladend

Ziel: Ein zusätzlicher Zustand, der in einem engen oberen SOC-Bereich (z.B. 95–99%) nur noch mit +2% pro Zyklus lädt, und bei 100% in NichtLadend übergeht. Außerdem sinnvolle Übergänge aus NormalLadend in Erhaltungsladend.

Listing 7: Neuer Zustand Erhaltungsladend

```
1 package _02_03_2026;
2
3 public class Erhaltungsladend extends Zustand {
4
5     private static final Erhaltungsladend INSTANCE = new
6         Erhaltungsladend();
7
8     private Erhaltungsladend() {
9         super();
10    }
11
12    public static Erhaltungsladend getErhaltungsladend() {
13        return INSTANCE;
14    }
15
16    @Override
17    public void bearbeiten(Ladegeraet ladegeraet) {
18        int alt = ladegeraet.getLadestand();
19        // Nur geringer Zuwachs zur Schonung des Akkus
20        ladegeraet.setLadestand(alt + 2);
21        int neu = ladegeraet.getLadestand();
22        System.out.printf("[Erhaltungsladend] %d%% -> %d%%\n", alt,
23            neu);
24
25        // Bei 100% aufhoeren und zu NichtLadend wechseln
26        if (neu >= 100) {
27            ladegeraet.setZustand(NichtLadend.getNichtLadend());
28            System.out.println(" -> 100% erreicht, Wechsel zu
29                NichtLadend");
30        }
31    }
32 }
```

Anpassungen in bestehenden Zuständen (Übergänge): - In NormalLadend.bearbeiten(...) nach dem Erhöhen prüfen: - Wenn neu >= 100 → NichtLadend - sonst, falls neu >= 95 → Erhaltungsladend

Listing 8: Ergänzung in NormalLadend

```
1 @Override
2 public void bearbeiten(Ladegeraet ladegeraet) {
3     int alt = ladegeraet.getLadestand();
4     ladegeraet.setLadestand(alt + 10);
5     int neu = ladegeraet.getLadestand();
6     System.out.printf("[NormalLadend] %d%% -> %d%%\n", alt, neu);
7 }
```



```

8         if (neu >= 100) {
9             ladegeraet.setZustand(NichtLadend.getNichtLadend());
10            System.out.println(" -> Akku voll, Wechsel zu NichtLadend");
11        } else if (neu >= 95) {
12            ladegeraet.setZustand(Erhaltungsladend.getErhaltungsladend()
13                                );
14            System.out.println(" -> >=95%, Wechsel zu Erhaltungsladend")
15            ;
16        }
17    }
18 }

```

Optional: In NichtLadend könntest du bei sehr hohem, aber nicht vollem SOC (z. B. 95–99%) statt NormalLadend direkt Erhaltungsladend setzen.

Aufgabe 2: Polymorphie beim Aufruf ausloesen() (Kurzlösung)

Stichpunkte zur Musterlösung:

- **Compile-Zeit:** Der Typ des Feldes `zustand` ist die *abstrakte* Klasse `Zustand`. Der Compiler weiß nur, dass es `zustand.bearbeiten(Ladegeraet)` gibt.
- **Laufzeit:** Das *tatsächlich* referenzierte Objekt (z. B. `NichtLadend`, `NormalLadend`, `SchnellLadend`, `Erhaltungsladend`) bestimmt, *welche* `bearbeiten(...)`-Implementierung ausgeführt wird.
- **Dynamisches Binden:** Die Methodenbindung geschieht zur Laufzeit (spätes Binden). Dadurch wechselt das Verhalten automatisch mit dem Zustand, ohne `if/switch`-Ketten.
- **State-Pattern-Effekt:** Das Ladegerät muss die konkreten Klassen nicht kennen. Neue Zustände lassen sich hinzufügen, ohne Aufrufer anzupassen (Open-Closed-Prinzip).

Aufgabe 3: Privater Konstruktor & Factory-Idee beim Singleton (Kurzlösung)

Warum private Konstruktor?

- Verhindert, dass von außen mit `new` beliebig viele Instanzen erzeugt werden.
- Hält die Kontrolle über die Erzeugung innerhalb der Klasse (Invariante „genau eine Instanz“).

Wie liefert eine statische Getter-Methode dennoch eine Instanz?

- Die Klasse besitzt ein `private static` Feld, das die einzigartige Instanz hält.
- Die **statische** Methode (z. B. `getNormalLadend()`) gibt diese Instanz zurück.
- Varianten:
 - *Eager:* Instanz wird beim Laden der Klasse erstellt (einfach, „thread-sicher genug“ für Demos).
 - *Lazy:* Instanz wird beim ersten Aufruf erzeugt (ggf. Synchronisation nötig für Thread-Sicherheit).

Vorteile und mögliche Nachteile:

- **Vorteile:** zentrale Instanz, kontrollierte Lebensdauer, geringer Overhead, globaler Zugriff über `ClassName.getInstance()`.
- **Nachteile (vorsichtig ansprechen):** enge Kopplung/globale Abhängigkeit kann Testbarkeit beeinträchtigen; bei Multithreading auf korrekte Implementierung achten; in großen Systemen besser mit Dependency Injection arbeiten.